

GENCYPHER CTF

Team Cyclone X

Methodology

Caught in 4K [CTF] [EASY]

Step 1: Identify what the file is

Run this to confirm the executable type:

```
file checker.exe
```

You should see something like:

```
PE32 executable (console) Intel 80386, for MS Windows
```

This confirms it is a Windows PE .exe, so static analysis tools will work.

Step 2: Pull out all readable strings

Extract printable text stored inside the binary:

```
strings checker.exe
```

This output is long, so filter only flag-related parts:

```
strings checker.exe | grep genzipher
```

Step 3: Note the candidate flags

From the filtered output, you will see something like:

```
genzipher{n0t_th3_r34l_fl4g}  
genzipher{h4rdc0d3d_f0r_y0ur_3y3s}
```

This shows the flag is embedded as plain text inside the executable.

Step 4: Use the challenge hint to pick the correct one

The challenge says the secret was “deleted” but still remains in memory. That means the correct flag is likely the one that looks like leftover hardcoded data, not the obvious fake.

So:

genzipher{n0t_th3_r34l_fl4g} is a decoy

genzipher{h4rdc0d3d_f0r_y0ur_3y3s} matches the hint and is the real one

Step 5: Confirm it is stored in read-only memory (optional proof)

Dump the read-only data section where such strings are stored:

```
objdump -s -j .rdata checker.exe
```

You should be able to spot the same flag text inside that dump.

Step 6: Print it in the required format

```
print("genzipher{h4rdc0d3d_f0r_y0ur_3y3s}")
```

Gatekeeper [CTF] [EASY]

Step 1: Read the challenge hint

The description says:

“Speak the language of opposites”

“It is merely a reflection”

In CTFs, this strongly points to XOR, because XOR flips bits and is often described as working with opposites.

Step 2: Inspect the file

The downloaded file is a small program that checks your input against a hidden value.

When you open it in a tool like:

strings

a basic disassembler

or even by reading the logic in a decompiler

you can see that the program does not generate the flag dynamically.

Instead, it:

Stores some encoded data

Applies XOR to it

Compares the result with your input

Step 3: Identify the XOR key

From the code logic, it becomes clear that:

A fixed XOR key is used

Each character is XORed with that key

Reversing the operation (XOR again with the same key) reveals the original text

This matches the hint:

“To the wise, it is merely a reflection”
(XOR is its own inverse)

Step 4: Decode the hidden value

Once you XOR the encoded bytes with the same key, the readable text appears:

```
genzipher{x0r_1s_th3_k3y_t0_th3_g4t3}
```

Step 5: Print the flag

Since the challenge only checks for a string match, the solution script is simply:

```
print("genzipher{x0r_1s_th3_k3y_t0_th3_g4t3}")
```

Trust Issues [CTF] [EASY]

The challenge hint says the secret is hidden in the “smallest bits”. That usually means the data is hidden inside the image file itself, not visible on the screen.

The image was downloaded and checked, but nothing unusual was visible. So the next step was to inspect the file contents.

The image file was opened using a basic tool like strings or a hex viewer to look for readable text hidden inside the file.

While scrolling through the extracted text, a string that matches the flag format `genzipher{...}` appeared.

Since the challenge checks the flag using a simple string comparison, the visible flag inside the file is the correct answer.

The flag was then printed using a simple Python `print()` statement, as required.

Void Walker [CTF] [HARD]

Download the `void_walker` file.

Check the file type to see what it is:

file `void_walker`

It is a Windows executable.

Since the challenge says the flag is checked using a simple string comparison, the flag must be inside the file.

Extract readable text from the binary:

```
strings void_walker
```

Search for the flag format:

```
strings void_walker | grep genzipher
```

The flag appears directly in the output:

```
genzipher{w1nd0ws_of_th3_v01d_w4lk3r}
```

Print it as required:

```
print("genzipher{w1nd0ws_of_th3_v01d_w4lk3r}")
```

Rookie Mistake Part 1 [CTF] [MEDIUM]

Step 1: Identify the Challenge Type

The provided file (decrypt.exe) is a compiled executable, indicating a reverse-engineering challenge. The problem description mentions bits, rhythm, and counting, which suggests a logic flaw in a cryptographic or bitwise operation, rather than brute force.

Step 2: Inspect the Binary for Encryption Logic

Basic inspection shows the program contains:

- A hardcoded encrypted byte array

- A fixed key

- A fixed mask

- A loop that performs XOR operations

This confirms the flag is encrypted inside the binary, not dynamically generated.

Step 3: Understand the Decryption Algorithm

By analyzing the program logic, the following behavior was identified:

The program decrypts the flag byte-by-byte

Each byte is XORed with:

A value from the mask

A value from the key

The loop processes the buffer sequentially

XOR encryption is reversible, so correct indexing is critical.

Step 4: Detect the Rookie Mistake

While reviewing the loop, an off-by-one indexing error was identified:

The loop index is shifted by one position

The mask is applied to the wrong byte

This causes every decrypted byte to be slightly incorrect

This type of error is a very common beginner mistake in cryptographic loops.

Step 5: Correct the Indexing Error

After fixing the index alignment so that:

The correct mask byte is applied to the correct data byte

The decryption logic produces a readable ASCII string instead of random data.

Step 6: Recover the Flag

Once the indexing bug is fixed, the decrypted output clearly reveals the flag:

```
genzipher{Off_by_One_bug_f0und}
```

Step 7: Submit the Flag

As required by the challenge, the solution was submitted using a simple script:

```
print("genzipher{Off_by_One_bug_f0und}")
```

Sands of Time

This binary verifies a secret string by transforming user input and comparing it to a hidden byte array. We reverse that process.

Find the verification function

Using symbols in the binary:

```
readelf -s sands_of_time | grep verify_timeline
```

We see:

verify_timeline

So this is the function that checks the password.

Disassemble the verifier

```
objdump -d -M intel sands_of_time --start-address=0x1222 --stop-address=0x12b9
```

Key logic inside:

```
mov eax, i
```

```
add eax, eax
```

```
add eax, i ; eax = 3*i
```

```
add eax, input[i]
```

```
xor eax, 0x42
```

```
cmp al, sands_secret[i]
```

So the transformation is:

$$\text{encoded}[i] = (\text{input}[i] + 3i) \oplus 0x42$$
$$\text{encoded}[i] = (\text{input}[i] + 3i) \oplus 0x42$$

Where $\oplus$ is XOR.

Extract the secret array

The comparison uses sands_secret, located at address 0x40a0.

Dump it:

```
objdump -s sands_of_time --start-address=0x40a0 --stop-address=0x40c6
```

We get 38 bytes:

25 2a 36 c1 c7 3d 38 38 c8 d4 d3 cb 17 cf dc 1c

dc d8 d7 ee 32 ef e4 fa e5 ff c3 85 c5 c9 84 fe

96 97 99 9f 96 ae

These are the encoded bytes.

Reverse the formula

From:

$encoded[i] = (input[i] + 3i) \oplus 0x42$
 $encoded[i] = (input[i] + 3i) \oplus 0x42$

We reverse it:

$input[i] = (encoded[i] \oplus 0x42) - 3i$
 $input[i] = (encoded[i] \oplus 0x42) - 3i$

Decode with script

PowerShell (what you used):

```
$bytes = 0x25,0x2a,0x36,0xc1,0xc7,0x3d,0x38,0x38,0xc8,0xd4,0xd3,0xcb,  
         0x17,0xcf,0xdc,0x1c,0xdc,0xd8,0xd7,0xee,0x32,0xef,0xe4,0xfa,  
         0xe5,0xff,0xc3,0x85,0xc5,0xc9,0x84,0xfe,0x96,0x97,0x99,0x9f,  
         0x96,0xae
```

```
$result = ""
```

```
for($i=0; $i -lt $bytes.Length; $i++){
```

```
    $val = (($bytes[$i] -bxor 0x42) - (3*$i)) -band 0xFF
```

```
    $result += [char]$val
```

```
}
```

```
$result
```

Output:

```
genzipher{sh1ft1ng_s4nds_r3v34l_truth}
```

67 [CTF] [HARD]

Solution Methodology

Step 1: Identify the Encoding Scheme

Initial Analysis:

The file contains only two characters: '6' and '7'

This pattern suggests a binary encoding scheme

Hypothesis: '6' represents binary '0' and '7' represents binary '1'

Action Taken:

Converted all '6' characters to '0' and all '7' characters to '1'

Grouped the resulting binary string into 8-bit chunks (bytes)

Converted each 8-bit chunk from binary to its ASCII/byte value

Saved the decoded output as a new file

Result:

The decoded output was approximately 34KB of heavily obfuscated JavaScript code.

Step 2: Execute and Observe Behavior

Initial Execution:

Ran the decoded JavaScript file using Node.js

Observed the program's output and behavior

Observations:

The script printed various system messages containing "67" throughout

Messages included initialization steps, checksums, and diagnostics

No flag was visible in the standard output

The script included artificial delays (6700ms = 6.7 seconds)

Conclusion:

The flag is not directly printed by the script - it must be hidden within the code's logic.

Step 3: Analyze Code Structure for String Construction

Search Strategy:

Examined the obfuscated JavaScript for common patterns used to build or hide strings:

Patterns Searched:

fromCharCode - Commonly used to construct strings from character codes

charCodeAt - Extracts character codes from strings

^ (XOR operator) - Frequently used in encryption/obfuscation

=== or == - Comparison operations that might validate flags

Buffer operations - Binary data manipulation

Variable concatenation patterns

Key Discovery:

Found multiple Buffer operations creating string variables:

Variable n created from hex-encoded buffer

Variable e created from hex-encoded buffer

Variable d created from hex-encoded buffer

Variable r created from hex-encoded buffer

Variable u created from hex-encoded buffer

These five variables were then concatenated into a final variable S.

Step 4: Identify the Hidden Flag Location

Critical Insight:

The flag is dynamically constructed at runtime by concatenating multiple buffers, not stored as plaintext.

Evidence:

Variable S is assigned the concatenation of n + e + d + r + u

This concatenated string is never printed to console

The variable S is created but then unused in output statements

Why Previous Attempts Failed:

Searching the decoded bytes for "genzipher{" found nothing (flag wasn't in plaintext)

Running the script normally didn't reveal the flag (it was never printed)

The flag was only held in memory during execution, not stored statically

Step 5: Patch the Code to Reveal the Flag

Patching Strategy:

Locate the exact line where variable S is assigned

Insert a console.log statement immediately after this assignment

Optionally remove delay mechanisms to speed up execution

Modification Details:

Found the line: `const S = [concatenation of n,e,d,r,u]`

Added after it: Print the value of S to console with a clear label

Changed delay value from 6700 milliseconds to 0 for faster execution

Step 6: Execute Patched Code

Final Execution:

Ran the modified JavaScript file

The script now printed the dynamically-constructed flag

Success:

=== FLAG FOUND ===

genzipher{s1xs3v3n_67_67_67_67_67}